Laboratory Exercise 7: complete solution

DE1-SoC board | Quartus Prime 18.1 | traditional Verilog-2001

This solution follows the attached lab7_Verilog.pdf. The previous counter/RAM lab is not used as a specification. All hardware source files use traditional Verilog, explicit vector widths, sized constants, named port connections, and nonblocking assignments for registers.

The designs and simulations are complete. Quartus compilation, the actual Quartus state-encoding comparison, and physical board observations still need to be performed on your computer and board. Expected results below are predictions, not invented measurements.

Files and projects

Open a project from the quartus folder. Keep the extracted directory structure intact because project source paths are relative.

Lab requirement	Source file	Quartus project	Top-level module
Part I, Table 1	rtl/part1.v	quartus/part1/part1.qpf	part1
Part I, Table 2 modification	rtl/part1_modified.v	quartus/part1_modified/part1_modified.qpf	part1_modified
Part II, binary encoding	rtl/part2.v	quartus/part2/part2.qpf	part2
Part II, encoding comparison	same rtl/part2.v	quartus/part2_one_hot/part2_one_hot.qpf	part2
Part III, two shift registers	rtl/part3.v	quartus/part3/part3.qpf	part3
Part III, optional one-register answer	rtl/part3_single.v	simulated with the other detectors	sequence_single_shift
Part IV, Morse encoder	rtl/part4.v	quartus/part4/part4.qpf	part4

Only add the relevant hardware file to a new Quartus project. Testbench files belong in the simulator, not in the synthesizable project. Each main source file contains its top-level board wrapper and the modules it needs; no generated IP is required.

Shared specification for Parts I and II

The input is w. Output z becomes 1 immediately after the fourth consecutive sampled 0 or 1. It stays 1 while that run continues. A change of input starts a new run of length one and makes z return to 0 after that clock edge. This is a Moore machine: z depends on the stored state.

State	Meaning	Next state if w=0	Next state if w=1	z
A	Reset; no samples yet	B	F	0
B	One consecutive 0	C	F	0
C	Two consecutive 0s	D	F	0
D	Three consecutive 0s	E	F	0
E	At least four consecutive 0s	E	F	1
F	One consecutive 1	B	G	0
G	Two consecutive 1s	B	H	0
H	Three consecutive 1s	B	I	0
I	At least four consecutive 1s	B	I	1

SW[0] is an active-low synchronous reset. SW[1] is w. KEY[0] is the manual clock. LEDR[9] is z. The state or shift-register contents occupy the lower LEDs as described for each part.

KEY[0] is 0 while pressed and 1 while released. Because the code uses posedge KEY[0], the sampling edge occurs when you release the button. Set the switches first, then press and release KEY0. To reset, set SW0=0 and release KEY0 once; setting SW0=0 alone does not change a synchronous state register. Return SW0 to 1 before applying data samples.

Part I: manually derived one-hot machine

The state bits y8 through y0 map to I, H, G, F, E, D, C, B, A. A has code 000000001. For every next-state equation, collect the states with arrows entering its destination and include the required input condition.

The following expressions give the D inputs during normal operation. The clocked register selects the reset code when reset_n=0.

```
D0 = 0
D1 = ~w & (y0 | y5 | y6 | y7 | y8)
D2 = ~w & y1
D3 = ~w & y2
D4 = ~w & (y3 | y4)
D5 =  w & (y0 | y1 | y2 | y3 | y4)
D6 =  w & y5
D7 =  w & y6
D8 =  w & (y7 | y8)
z  = y4 | y8
```

For example, state B is entered from A or from any of F-I when the next input is 0. That gives D1. State E is entered from D or stays in E when the input is 0. That gives D4.

rtl/part1.v uses only continuous assign expressions for the normal next-state logic. A 9-bit reg updated in an always @(posedge clk) block describes the nine state flip-flops. Its ternary expression selects the reset value or the D-input vector; it is not a procedural state table.

LEDR[8:0] shows y_Q. In normal legal operation exactly one of these LEDs is on. LEDR9 turns on in E or I. Reset selects A, so LEDR0 is on after a reset clock edge.

Part I modification: reset code with all zeros

Invert the original y0 bit; leave the other eight bits unchanged. In this version, y0=0 in A and y0=1 in every other legal state.

State	Original y8...y0	Modified y8...y0
A	000000001	000000000
B	000000010	000000011
C	000000100	000000101
D	000001000	000001001
E	000010000	000010001
F	000100000	000100001
G	001000000	001000001
H	010000000	010000001
I	100000000	100000001

Make these changes in rtl/part1_modified.v:

- Reset the register to 9'b000000000.
- Change the normal D0 expression from 1'b0 to 1'b1.
- Replace y_Q[0] with ~y_Q[0] in D1 and D5.
- Keep all other equations, including z=y4|y8, unchanged.

This is the exact assignment in Table 2. Its non-reset codes have two 1s, although the lab calls it modified one-hot. The state sequence and z timing stay the same. All nine state LEDs are off in A; after leaving A, LEDR0 and the corresponding state LED are on.

Part II: case-based binary machine

Use the transition table above with the lab's binary codes A=0, B=1, ..., I=8. Four bits are sufficient for nine states.

rtl/part2.v has a combinational always @(*) case block assigning Y_D with blocking assignments, a separate clocked block assigning y_Q with nonblocking assignments, and an assign statement for z. Each case entry uses a clear two-input ternary selection. LEDR[3:0] displays the binary state; LEDR[8:4] is tied to zero.

The supplied skeleton uses default: Y_D = 4'bxxxx. This solution uses default: Y_D = A to define the RTL behavior for unused state codes and avoid propagating X values. This is a source-level choice; it is not a request for Quartus's separate Safe State Machine option.

Required Quartus encoding comparison

The part2 project sets STATE_MACHINE_PROCESSING to USER-ENCODED. The part2_one_hot project uses the identical Verilog source and sets that option to ONE-HOT.

In Quartus, the lab's route is Assignments > Settings > Compiler Settings > Advanced Settings (Synthesis) > State Machine Processing. Depending on the dialog layout, this may appear under Analysis & Synthesis Settings. Select User-Encoded first and compile.

Open Tools > Netlist Viewers > RTL Viewer and inspect the inferred FSM. Verify the nine states and both transitions from each state. In Compilation Report > Analysis & Synthesis > State Machines, verify the binary encoding. Capture this report before changing the setting.

Recompile using One-Hot and inspect the same report. Compare internal state-register codes with Table 2. Quartus commonly inverts the reset-state bit, producing an all-zero reset state and two set bits in each other state. This has the same principle as Table 2. Bit ordering or the association between a non-reset state and a particular bit may differ. Use the actual report to identify the differences; the exact compiled ordering is not known before running Quartus.

Suggested discussion after observing your report: "User-Encoded preserved the specified binary state assignment. One-Hot changed the internal state representation while retaining the same transitions and output behavior. Its all-zero reset code and inverted reset-state bit are consistent with the modified one-hot scheme. Any differences in bit ordering are encoding choices and do not change the function."

The LEDs are HDL outputs, not a guaranteed direct display of a recoded internal register. If Quartus recodes the FSM, it may add output decoding to preserve the original four-bit state output. Use the State Machines report to compare internal encodings.

Part III: two shift-register detector

The required implementation is rtl/part3.v. One four-bit register shifts in w; the other shifts in ~w. A stored 1 means the corresponding sample matched that register's target input.

```
ones  <= {ones[2:0], w};
zeros <= {zeros[2:0], ~w};
assign z = (&ones) | (&zeros);
```

Reset both registers to 0000. This is important: neither register initially contains four valid matches. Four actual matching samples are needed before its reduction AND becomes 1.

LEDR[3:0] shows the ones register, LEDR[7:4] shows the zeros register, LEDR8 is off, and LEDR9 is z. After four 0 samples, zeros=1111 and ones=0000. After four 1 samples, ones=1111 and zeros=0000. Overlap works naturally because the registers always hold a rolling window of samples.

Answer: could one four-bit shift register be used?

Yes. Store the last four raw values of w in one register called history. Detect 1111 or 0000:

```
assign z = (samples == 3'd4) & ((&history) | (~|history));
```

The reduction AND tests for all ones. The reduction NOR tests for all zeros. A saturating sample counter ensures the window contains four real input samples after reset. rtl/part3_single.v supplies this optional implementation: one four-bit shift register plus a three-bit counter, for seven RTL storage bits rather than the two-register design's eight.

An ordinary single history register alone is not sufficient to guarantee correct startup immediately after reset. Resetting it to 0000 would indicate four zeros before any samples arrive. A mixed reset pattern also cannot solve every startup case: after three equal real samples, the one remaining reset bit can match them and create an early detection. The validity counter removes this problem.

Part IV: Morse-code encoder

Use CLOCK_50 as the system clock, SW[2:0] to select A-H, KEY1 to start once, KEY0 as an active-low asynchronous reset, and LEDR0 as the Morse output. The other LEDs are off.

The lab specifies dot and dash durations but does not specify the gap or behavior of another start press during transmission. This solution chooses a 0.5-second LED-off gap between symbols, ignores presses while busy, and requires a new press to

start another transmission. It sends one selected letter per press and stops after the final symbol. Changing the switches during transmission does not alter the already loaded letter. Set them before pressing KEY1.

Letter lookup

In the four-bit code, 0 means dot and 1 means dash. Bit 3 is sent first; unused low bits are padding. The length counter distinguishes, for example, E from H despite both having a 0000 pattern.

SW[2:0]	Letter	Morse	Code [3:0]	Length	Total transmit time
000	A	.-	0100	2	2.5 s
001	B	-...	1000	4	4.5 s
010	C	-.-.	1010	4	5.5 s
011	D	-..	1000	3	3.5 s
100	E	.	0000	1	0.5 s
101	F	..-.	0010	4	4.5 s
110	G	--.	1100	3	4.5 s
111	H	....	0000	4	3.5 s

Times include the inter-symbol gaps, starting at the first LED-on edge and ending at the final LED-off edge. The few system-clock cycles used to synchronize a start press are outside these times.

Timer and datapath

50,000,000 clock cycles per second times 0.5 seconds = 25,000,000 cycles. A 25-bit counter counts from 0 through 24,999,999. At the terminal count it asserts tick, which acts as a clock enable. All registers still use posedge CLOCK_50; no derived clock is used.

morse_letter is the combinational lookup table. morse is the four-bit shift register. remaining is the three-bit symbol count. Both load on an accepted press. At the end of each gap, morse shifts left and remaining decreases by one. The timer is cleared in IDLE so the first pulse always begins with a complete timing interval.

KEY1 goes through two synchronizing flip-flops and a previous-value register. A falling-edge detector generates one start event rather than repeatedly transmitting while the button is held. KEY0 resets the controller, timer, datapath, and start detector asynchronously.

Controller state table

Each non-IDLE state holds until tick. LEDR0 is on in PULSE1, PULSE2, and PULSE3, and off in IDLE and GAP.

State	Action or transition at tick	LED
IDLE	On a start event, load code/length and enter PULSE1	Off
PULSE1	Dash: PULSE2. Dot: IDLE if last symbol, otherwise GAP	On
PULSE2	Enter PULSE3	On
PULSE3	IDLE if last symbol, otherwise GAP	On
GAP	Shift code, decrement remaining, enter PULSE1	Off

A dot stays high for one interval. A dash stays continuously high through three intervals; it does not go low between PULSE1, PULSE2, and PULSE3. No trailing timed gap is inserted after the final symbol; the encoder returns to IDLE with the LED off.

For A, the visible sequence is on for 0.5 s, off for 0.5 s, on for 1.5 s, then off until another start press. The complete synthesizable code is rtl/part4.v and is reproduced in the appendix.

Quartus and board procedure

1. Extract the package. Open the appropriate .qpf from the project table. The projects target Cyclone V 5CSEMA5F31C6 and include the relevant pins, source file, clock constraint, and encoding setting.

2. If making your own project, select that device, add the matching rtl file, and set the matching top-level module. Import the pin assignments from the supplied project or use the table below.

3. Compile with Processing > Start Compilation. Check errors and inspect the RTL Viewer. For Part II, inspect and record both FSM encoding reports.

4. Run the supplied simulation before programming. In Tools > Programmer, choose USB-Blaster II and load the compiled .sof on the FPGA device in the JTAG chain. Select Program/Configure and Start.

5. For Parts I-III, reset with SW0=0 and one KEY0 release, then set SW0=1. Apply SW1 data with one press-release per sample.

6. For Part IV, press KEY0 to reset, release it, select a letter on SW2-SW0, and press KEY1. Observe LEDR0. Use all eight letters to verify the lookup.

The .sdc files constrain the clock. Manual-clock projects use a conservative 20 ns internal timing target even though your manual button edges are much slower. These are educational clock constraints, not a complete specification of timing for asynchronous user switches and LEDs. Keep the data switches stable before and around a manual sampling edge. The Part IV project uses a real 20 ns CLOCK_50 constraint.

FPGA pins

These pins follow Terasic's DE1-SoC user manual, Tables 3-5 through 3-8. The FPGA LEDs are active high; the pushbuttons are active low. Use the board's existing button conditioning for this lab. Check the manual corresponding to your board revision when setting up your hardware.

Signal	FPGA pin	Signal	FPGA pin
CLOCK_50	AF14	LEDR[0]	V16
KEY[0]	AA14	LEDR[1]	W16
KEY[1]	AA15	LEDR[2]	V17
SW[0]	AB12	LEDR[3]	V18
SW[1]	AC12	LEDR[4]	W17
SW[2]	AF9	LEDR[5]	W19
LEDR[6]	Y19	LEDR[7]	W20
LEDR[8]	W21	LEDR[9]	Y21

The .qsf files use 3.3-V LVTTL for these ports. Only ports declared by the selected top-level module are assigned.

Simulation and expected observations

With ModelSim-Altera, change the working directory to the package's tb folder. In the Transcript, run do sim_sequence.do for Parts I-III or do sim_morse.do for Part IV. The scripts compile the relevant sources and run the testbench. PASS messages indicate success; FAIL messages describe a mismatch. After a simulation ends, use quit -sim before launching the other one if needed.

If Icarus Verilog is installed, run python run_tests.py from the package folder. The runner checks the PASS/FAIL messages as well as the command exit status.

The Morse testbench deliberately overrides HALF_SECOND_CYCLES with 8 to make simulation fast. Keep the default 25'd25000000 in the board design.

A manual sequence test

This trace applies to both FSM encodings and the shift-register detector. Each row is after a rising clock edge; reset is released before the first input sample.

Sample	Input w	Expected state	z
Reset	ignored	A	0
1	0	B	0
2	0	C	0
3	0	D	0

Sample	Input w	Expected state	z
4	0	E	1
5	0	E	1
6	1	F	0
7	1	G	0
8	1	H	0
9	1	I	1
10	1	I	1
11	0	B	0

Also test an alternating input sequence; z should remain 0. Reset after a detected run; z should return to 0 on the next manual clock edge in Parts I-III. In Part IV, reset should turn the LED off immediately, even in the middle of a dash.

Verification performed

- Icarus Verilog 12.0 compiled/elaborated each of the five board top-level modules in Verilog-2001 mode without diagnostics.

- The sequence test checked all 4,096 twelve-bit sequences from reset, totaling 49,152 sequence samples, plus 64 samples in long runs: 49,216 input samples overall. It compared state codes and outputs across original one-hot, modified one-hot, binary, two-shift-register, and optional one-shift-register implementations.

- The sequence test also checked that synchronous reset does not act between clock edges and that reset outputs match the specified encodings.

- The Morse test checked all eight letters and all 25 symbols, exact high and inter-symbol low durations, switch changes after loading, held start, a new start press while busy, restart after release, and asynchronous reset during a dash.

- The simulator's half-second unit was shortened to eight cycles. The 0.5 s and 1.5 s hardware durations follow from the default count and 50 MHz input; they were not measured on a physical board.

verification/results.txt records the simulation results. Quartus-generated state ordering, resource use, timing closure, and physical LED observations remain local lab checks. Do not present the expected results as measurements until you run those steps.

References

- Assignment: supplied lab7_Verilog.pdf, Laboratory Exercise 7, Finite State Machines, Parts I-IV.

- Board hardware: Terasic DE1-SoC User Manual, January 28, 2019, clock and FPGA I/O tables. Author's resources: https://www.terasic.com.cn/cgi-bin/page/archive.pl?CategoryNo=165&Language=English&No=836&PartNo=4

- The Terasic-authored manual is also readable at: https://studylib.net/doc/25261674/de1-soc-user-manual-revf

- Quartus options: https://resources.altera.com/quartushelp/17.0/comp/comp/comp_tab_fitting.htm

- Quartus state-encoding behavior: Intel Quartus Prime Standard Edition User Guide, Design Compilation, State Machine Processing. Search-indexed source: https://www.intel.com/programmable/technical-pdfs/qps-ugs.pdf (the current redirect may be unavailable).

Appendix: complete hardware source

The following files are also available individually in the rtl folder. All simulation code is in tb.

part1.v

```
module part1 (
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
);
    wire [8:0] state;
    wire z;

    sequence_one_hot detector (
        .clk(KEY[0]),
        .reset_n(SW[0]),
        .w(SW[1]),
        .state(state),
        .z(z)
    );

    assign LEDR[8:0] = state;
    assign LEDR[9] = z;
endmodule

module sequence_one_hot (
    input clk,
    input reset_n,
    input w,
    output [8:0] state,
    output z
);
    reg [8:0] y_Q;
    wire [8:0] Y_D;

    // A=y0, B=y1, C=y2, D=y3, E=y4, F=y5, G=y6, H=y7, I=y8.
    assign Y_D[0] = 1'b0;
    assign Y_D[1] = ~w & (y_Q[0] | y_Q[5] | y_Q[6] | y_Q[7] | y_Q[8]);
    assign Y_D[2] = ~w & y_Q[1];
    assign Y_D[3] = ~w & y_Q[2];
    assign Y_D[4] = ~w & (y_Q[3] | y_Q[4]);
    assign Y_D[5] = w & (y_Q[0] | y_Q[1] | y_Q[2] | y_Q[3] | y_Q[4]);
    assign Y_D[6] = w & y_Q[5];
    assign Y_D[7] = w & y_Q[6];
    assign Y_D[8] = w & (y_Q[7] | y_Q[8]);

    // This 9-bit register describes nine flip-flops. Reset has priority.
    always @(posedge clk) begin
        y_Q <= !reset_n ? 9'b000000001 : Y_D;
    end

    assign state = y_Q;
    assign z = y_Q[4] | y_Q[8];
endmodule
```

part1_modified.v

```
module part1_modified (
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
);
    wire [8:0] state;
    wire z;

    sequence_modified_one_hot detector (
        .clk(KEY[0]),
        .reset_n(SW[0]),
        .w(SW[1]),
        .state(state),
        .z(z)
    );

    assign LEDR[8:0] = state;
    assign LEDR[9] = z;
endmodule

module sequence_modified_one_hot (
    input clk,
    input reset_n,
    input w,
    output [8:0] state,
    output z
);
    reg [8:0] y_Q;
    wire [8:0] Y_D;

    // Table 2: y0 is inverted; A is now 000000000.
    assign Y_D[0] = 1'b1;
    assign Y_D[1] = ~w & (~y_Q[0] | y_Q[5] | y_Q[6] | y_Q[7] | y_Q[8]);
    assign Y_D[2] = ~w & y_Q[1];
    assign Y_D[3] = ~w & y_Q[2];
    assign Y_D[4] = ~w & (y_Q[3] | y_Q[4]);
    assign Y_D[5] = w & (~y_Q[0] | y_Q[1] | y_Q[2] | y_Q[3] | y_Q[4]);
    assign Y_D[6] = w & y_Q[5];
    assign Y_D[7] = w & y_Q[6];
    assign Y_D[8] = w & (y_Q[7] | y_Q[8]);

    always @(posedge clk) begin
        y_Q <= !reset_n ? 9'b000000000 : Y_D;
    end

    assign state = y_Q;
    assign z = y_Q[4] | y_Q[8];
endmodule
```

part2.v

```
module part2 (
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
);
    wire [3:0] state;
    wire z;

    sequence_binary detector (
        .clk(KEY[0]),
        .reset_n(SW[0]),
        .w(SW[1]),
        .state(state),
        .z(z)
    );

    assign LEDR[3:0] = state;
    assign LEDR[8:4] = 5'b00000;
    assign LEDR[9] = z;
endmodule

module sequence_binary (
    input clk,
    input reset_n,
    input w,
    output [3:0] state,
    output z
);
    reg [3:0] y_Q;
    reg [3:0] Y_D;

    parameter A = 4'b0000, B = 4'b0001, C = 4'b0010,
              D = 4'b0011, E = 4'b0100, F = 4'b0101,
              G = 4'b0110, H = 4'b0111, I = 4'b1000;

    always @(*) begin
        case (y_Q)
            A: Y_D = w ? F : B;
            B: Y_D = w ? F : C;
            C: Y_D = w ? F : D;
            D: Y_D = w ? F : E;
            E: Y_D = w ? F : E;
            F: Y_D = w ? G : B;
            G: Y_D = w ? H : B;
            H: Y_D = w ? I : B;
            I: Y_D = w ? I : B;
            default: Y_D = A;
        endcase
    end

    // Active-low synchronous reset, as required in Parts I and II.
    always @(posedge clk) begin
        y_Q <= !reset_n ? A : Y_D;
    end

    assign state = y_Q;
    assign z = (y_Q == E) | (y_Q == I);
endmodule
```

part3.v

```
module part3 (
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
);
    wire [3:0] ones;
    wire [3:0] zeros;
    wire z;

    sequence_shift detector (
        .clk(KEY[0]),
        .reset_n(SW[0]),
        .w(SW[1]),
        .ones(ones),
        .zeros(zeros),
        .z(z)
    );

    assign LEDR[3:0] = ones;
    assign LEDR[7:4] = zeros;
    assign LEDR[8] = 1'b0;
    assign LEDR[9] = z;
endmodule

module sequence_shift (
    input clk,
    input reset_n,
    input w,
    output reg [3:0] ones,
    output reg [3:0] zeros,
    output z
);
    // Each register collects matches: 1 for its target input, 0 otherwise.
    always @(posedge clk) begin
        ones  <= !reset_n ? 4'b0000 : {ones[2:0], w};
        zeros <= !reset_n ? 4'b0000 : {zeros[2:0], ~w};
    end

    // Reduction AND is true only when all four stored matches are 1.
    assign z = (&ones) | (&zeros);
endmodule
```

part3_single.v

```
// Optional answer to the Part III question; part3.v is the required design.
module sequence_single_shift (
    input clk,
    input reset_n,
    input w,
    output reg [3:0] history,
    output z
);
    reg [2:0] samples;

    // The window becomes valid after four real samples, not reset padding.
    always @(posedge clk) begin
        if (!reset_n) begin
            history <= 4'b0000;
            samples <= 3'd0;
        end else begin
            history <= {history[2:0], w};
            samples <= (samples < 3'd4) ? samples + 3'd1 : samples;
        end
    end

    assign z = (samples == 3'd4) & ((&history) | (~|history));
endmodule
```

part4.v

```
module part4 (
    input CLOCK_50,
    input [2:0] SW,
    input [1:0] KEY,
    output [9:0] LEDR
);
    wire morse_led;

    morse_encoder encoder (
        .clk(CLOCK_50),
        .reset_n(KEY[0]),
        .start_n(KEY[1]),
        .letter(SW[2:0]),
        .led(morse_led)
    );

    assign LEDR[0] = morse_led;
    assign LEDR[9:1] = 9'b000000000;
endmodule

module morse_letter (
    input [2:0] letter,
    output reg [3:0] code,
    output reg [2:0] length
);
    // Send bit 3 first: dot=0, dash=1; unused low bits are padding.
    always @(*) begin
        case (letter)
            3'd0: begin code = 4'b0100; length = 3'd2; end // A .-
            3'd1: begin code = 4'b1000; length = 3'd4; end // B -...
            3'd2: begin code = 4'b1010; length = 3'd4; end // C -.-.
            3'd3: begin code = 4'b1000; length = 3'd3; end // D -..
            3'd4: begin code = 4'b0000; length = 3'd1; end // E .
            3'd5: begin code = 4'b0010; length = 3'd4; end // F ..-.
            3'd6: begin code = 4'b1100; length = 3'd3; end // G --.
            3'd7: begin code = 4'b0000; length = 3'd4; end // H ....
            default: begin code = 4'b0000; length = 3'd1; end
        endcase
    end
endmodule

module half_second_timer #(
    parameter [24:0] HALF_SECOND_CYCLES = 25'd25000000
) (
    input clk,
    input reset_n,
    input clear,
    output tick
);
    reg [24:0] count;

    assign tick = (count == HALF_SECOND_CYCLES - 25'd1);

    // clear starts a fresh interval; tick is a clock enable, not a clock.
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            count <= 25'd0;
        else
            count <= (clear | tick) ? 25'd0 : count + 25'd1;
    end
endmodule

module morse_encoder #(
    parameter [24:0] HALF_SECOND_CYCLES = 25'd25000000
) (
    input clk,
    input reset_n,
    input start_n,
    input [2:0] letter,
    output led
);
    parameter IDLE = 3'd0, PULSE1 = 3'd1, PULSE2 = 3'd2,
```

```
          PULSE3 = 3'd3, GAP = 3'd4;

reg [2:0] state;
reg [2:0] next_state;
reg [3:0] morse;
reg [2:0] remaining;
reg start_meta;
reg start_sync;
reg start_previous;

wire [3:0] letter_code;
wire [2:0] letter_length;
wire start_press;
wire load;
wire shift;
wire tick;

morse_letter selection (
    .letter(letter),
    .code(letter_code),
    .length(letter_length)
);

half_second_timer #(
    .HALF_SECOND_CYCLES(HALF_SECOND_CYCLES)
) timer (
    .clk(clk),
    .reset_n(reset_n),
    .clear(state == IDLE),
    .tick(tick)
);

// Synchronize KEY1, then detect its active-low press once.
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        start_meta <= 1'b1;
        start_sync <= 1'b1;
        start_previous <= 1'b1;
    end else begin
        start_meta <= start_n;
        start_sync <= start_meta;
        start_previous <= start_sync;
    end
end

assign start_press = start_previous & ~start_sync;
assign load = (state == IDLE) & start_press;
assign shift = (state == GAP) & tick;

always @(*) begin
    next_state = state;
    case (state)
        IDLE: next_state = start_press ? PULSE1 : IDLE;
        PULSE1: begin
            if (tick) begin
                if (morse[3])
                    next_state = PULSE2;
                else
                    next_state = (remaining == 3'd1) ? IDLE : GAP;
            end
        end
        PULSE2: next_state = tick ? PULSE3 : PULSE2;
        PULSE3: begin
            if (tick)
                next_state = (remaining == 3'd1) ? IDLE : GAP;
        end
        GAP: next_state = tick ? PULSE1 : GAP;
        default: next_state = IDLE;
    endcase
end

// Load priority precedes shift. The selected letter is latched once.
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
```

```
            state <= IDLE;
            morse <= 4'b0000;
            remaining <= 3'd0;
        end else begin
            state <= next_state;
            if (load) begin
                morse <= letter_code;
                remaining <= letter_length;
            end else if (shift) begin
                morse <= {morse[2:0], 1'b0};
                remaining <= remaining - 3'd1;
            end
        end
    end

    // One high interval for a dot; three contiguous high intervals for a dash.
    assign led = (state == PULSE1) | (state == PULSE2) | (state == PULSE3);
endmodule
```